# Beyond the GPU: A Microarchitectural Analysis

## of LLM Tool-Calling Kernels

Kunpeng Zhang: kza63@sfu.ca
Fazal Rehman: fra22@sfu.ca
TJ Grewal: tga47@sfu.ca
*Simon Fraser University*

April 13, 2026

**Abstract**

As Large Language Models (LLMs) evolve from static chat interfaces into autonomous agents, the computational bottleneck is rapidly shifting from the GPU forward-pass to CPU-bound tool orchestration. Agentic workflows rely heavily on serial, branch-heavy, and I/O-bound tasks—such as JSON parsing, database lookups, and filesystem traversal—which behave more like traditional datacenter Remote Procedure Calls (RPC) than highly parallel machine learning kernels. In this paper, we introduce a decoupled microarchitectural profiling framework utilizing hardware performance counters (PAPI) to isolate and quantify the "Agentic Tax": the systemic overhead imposed by LLM orchestration frameworks. We evaluate compute, memory, and I/O-bound C++ execution kernels, demonstrating that Python-based orchestration and JSON serialization induce latency penalties up to 69.5× compared to native execution. Furthermore, we apply classical hardware optimizations—including SIMD vectorization (AVX2), software prefetching, and OpenMP concurrency—to the tool kernels. While software prefetching successfully yielded a 14.2% IPC improvement at the kernel level, the end-to-end system speedup remained below 1%. We term this phenomenon the Masking Effect: the execution bloat of the orchestration layer is so massive that it effectively nullifies microarchitectural hardware optimizations. Ultimately, we argue that the future of agentic AI requires treating the orchestration layer as a high-performance networking data plane.

## 1 Introduction

With the rapid proliferation of autonomous AI agents, the software industry is increasingly deploying these systems to automate complex engineering workflows. Historically, optimization efforts within the Generative AI ecosystem have overwhelmingly focused on accelerating the GPU forward-pass, minimizing Time-To-First-Token (TTFT) and inter-token latency [1,2]. However, as Large Language Models (LLMs) transition from passive text generators to active agents, the execution bottleneck shifts abruptly from the accelerator back to the host CPU [3].

Despite this growing reliance on tool-augmented LLMs, current literature often treats the tool-execution pipeline as a black box, measuring only high-level wall-clock latency without isolating the underlying hardware penalties [7]. To address this gap, this paper presents a first-principles microarchitectural characterization of the CPU-side execution pipeline. We designed a custom, decoupled benchmarking harness that strictly separates the Python-based LLM orchestrator from compiled, native C++ execution kernels.

Our contributions are threefold:

1. **Quantification of the Agentic Tax:** We demonstrate that the combined GPU/CPU overhead of LLM orchestration imposes a massive latency penalty, slowing down lightweight tools by up to 69.5× compared to native execution.

2. **Microarchitectural Characterization:** We map specific tool classes to their defining hardware bottlenecks, revealing how multi-step agentic loops compound CPU inefficiency through context accumulation.

3. **Identification of the Masking Effect:** We prove that while classical hardware optimizations (AVX2, prefetching) yield significant kernel-level efficiency gains, the massive framework bloat effectively "masks" these improvements, yielding negligible system-level speedups.

## 2 Background

To contextualize the microarchitectural bottlenecks evaluated in this study, we must define the paradigms of agentic execution and the structural limitations of modern accelerators.

### 2.1 LLM Orchestration and the ReAct Framework

Traditional LLMs operate as stateless generators. The advent of "Tool-Calling" transforms these models into agents, typically utilizing the ReAct (Reasoning and Acting) framework [10]. The model generates structured JSON strings to trigger external functions. An "Orchestration Layer"—such as a Python server—must intercept this generation, parse the JSON, execute the native subprocess, and inject the result back into the context window. This strictly serial cycle of serialization and context accumulation forms the backbone of agentic workflows [5].

### 2.2 Architectural Divergence: ML vs. Datacenter Workloads

Modern LLM performance is driven by GPU-accelerated matrix multiplications. However, agentic tool calls trigger tasks that are fundamentally hostile to GPU architectures [3]. Operations like querying a database or traversing a filesystem are inherently scalar, requiring deep recursive logic and complex memory pointer chasing. Consequently, they suffer from high Last-Level Cache (LLC) miss rates and branch mispredictions, making them behave more like traditional datacenter Remote Procedure Calls (RPC) than highly parallel machine learning kernels [1].

### 2.3 Hardware Telemetry via PAPI

To accurately measure the efficiency of native execution kernels without the noise of the Python runtime, systems researchers utilize Hardware Performance Counters (HPCs). By accessing the Linux Performance Monitoring Unit (PMU) via the Performance Application Programming Interface (PAPI), we achieve cycle-accurate telemetry of microarchitectural events, allowing us to isolate the 'Agentic Tax' at a hardware level.

## 3 Project Methodology

To accurately isolate and quantify the microarchitectural overhead of LLM tool orchestration (the "Agentic Tax"), we designed a custom, decoupled benchmarking harness. Our methodology strictly separates the orchestration layer from the execution layer, allowing us to capture high-fidelity hardware telemetry without cross-contamination between the Python runtime and the compiled C++ tool kernels.

### 3.1 System Architecture and Decoupling

Our system architecture is divided into three distinct components:

1. **The Orchestrator (Python/FastAPI):** An asynchronous server responsible for prompt assembly, tokenization, model inference, and JSON tool-call routing.

2. **The Execution Kernels (C++):** A suite of compiled, native binaries designed to execute specific bounded workloads (Compute, Memory, and I/O) without any awareness of the LLM.

3. **The Static Workloads:** Pre-generated, deterministic stress payloads ensuring run-to-run consistency across experiments.

By decoupling the architecture, we ensure that the C++ kernels are completely isolated from Python's Global Interpreter Lock (GIL) and memory-management overhead, establishing a pristine environment for microarchitectural profiling.

### 3.2 Microarchitectural Telemetry Integration

To capture low-level CPU behavior, we integrated the Performance Application Programming Interface (PAPI) via the `pypapi` C-bindings. We tap directly into the Linux kernel's Performance Monitoring Unit (PMU) to track four specific hardware events: Total Cycles (`PAPI_TOT_CYC`), Total Instructions (`PAPI_TOT_INS`), Branch Mispredictions (`PAPI_BR_MSP`), and Level 3 Cache Misses (`PAPI_L3_TCM`). All microarchitectural telemetry was captured on an

isolated testbed utilizing an Intel(R) Xeon(R) Gold 6242R processor running at a base frequency of 3.10GHz, ensuring predictable cycle-to-latency conversions.

To prevent idle GPU wait times from skewing the CPU metrics [6], we implemented a tightly scoped windowing strategy. PAPI hardware locks are acquired and released exactly around specific execution phases:

- **CPU Window 1 (Setup):** Captures prompt template assembly and tokenization.
- **CPU Window 2 (Serialization):** Captures output decoding, JSON extraction, and string manipulation.

The hardware EventSet is strictly managed within a `try...finally` block, ensuring that even if the agentic loop fails mid-execution due to hallucinated JSON, the Linux kernel registers are gracefully released, preventing PMU starvation.

### 3.3 The Agentic Execution Engine

We utilized `Qwen/Qwen2.5-7B-Instruct` as our base model, loaded in half-precision (FP16) utilizing Scaled Dot-Product Attention (SDPA) for optimized VRAM footprint. To force deterministic tool-calling behavior and eliminate variance between runs, generation sampling was disabled (`do_sample=False`).

Prior benchmarking reports a "parameter paradox" in tool-oriented workflows: protocol adherence does not monotonically improve with parameter count, and lightweight models can match or surpass larger models in structured tool-call compliance [7]. Nonetheless, LLMs may occasionally emit malformed tool-call syntax (e.g., multiple JSON objects or mixed natural language with JSON), which can break strict parsers. To mitigate this, our orchestration layer implements a robust regex-based extraction pipeline that isolates the first valid JSON object, executes the corresponding tool via a subprocess, and injects the `stdout` result back into the context window under the official Hugging Face `"tool"` role. This dynamic ReAct loop dynamically resizes its step count based on the model's autonomous decisions, capped at a hard limit to prevent infinite loops.

### 3.4 Tool Kernels and Workloads

To represent the diverse landscape of agentic operations, we developed three distinct C++ kernels, each targeting a different microarchitectural bottleneck. Corresponding Python generator scripts were built to synthesize massive, static payloads for these tools:

- **Compute-Bound (Calculator):** An Abstract Syntax Tree (AST) evaluator processing deeply nested mathematical expressions to stress branch prediction and instruction throughput.
- **Memory-Bound (Database Lookup):** A SQLite-based tool that executes forced full-table scans via `LIKE` queries to bypass indexing, intentionally thrashing the Last-Level Cache (LLC).
- **I/O-Bound (Directory Walker):** A recursive filesystem traversal tool that queries OS metadata to calculate total directory byte sizes, stressing context switches and system calls.

Crucially, each C++ tool manages its own internal PAPI profiling, tracking metrics only during the core workload execution and printing a standardized JSON result to `stdout`.

### 3.5 The Control Baseline Pipeline

To definitively prove the existence and scale of the "Agentic Tax," we implemented a `pipeline_mode="direct"` control pathway. In this mode, the GPU and LLM are bypassed entirely. The orchestrator uses regular expressions to extract target file paths directly from the user's prompt and executes the corresponding C++ kernels sequentially. By comparing the hardware metrics of this Direct Baseline against the Full Agentic Loop, we mathematically isolate the exact cycles, instructions, and cache misses consumed purely by LLM orchestration.

## 4 Evaluation Results

To address our core research questions, we present our findings in sequential phases. First, we quantify the macro-level "Agentic Tax" to understand the end-to-end latency penalty imposed by the LLM orchestrator. Subsequently, we dive into the microarchitectural telemetry to understand how this orchestration masks traditional hardware optimizations.

### 4.1 Quantifying the Agentic Tax (End-to-End Latency)

Before analyzing instruction throughput or cache behavior, we must establish the baseline orchestration penalty. We define the theoretical "Speed of Light" as our Direct Baseline (`pipeline_mode="direct"`), where prompts are routed to the compiled C++ execution kernels using static regular expressions, completely bypassing the GPU and LLM. We compare this against the Full Agentic Loop, where Qwen-2.5-7B autonomously parses the prompt, generates JSON tool calls, and orchestrates the execution.

Figure 1 illustrates the end-to-end request latency for all three bounded workloads (Compute, Memory, and I/O) on a logarithmic scale.

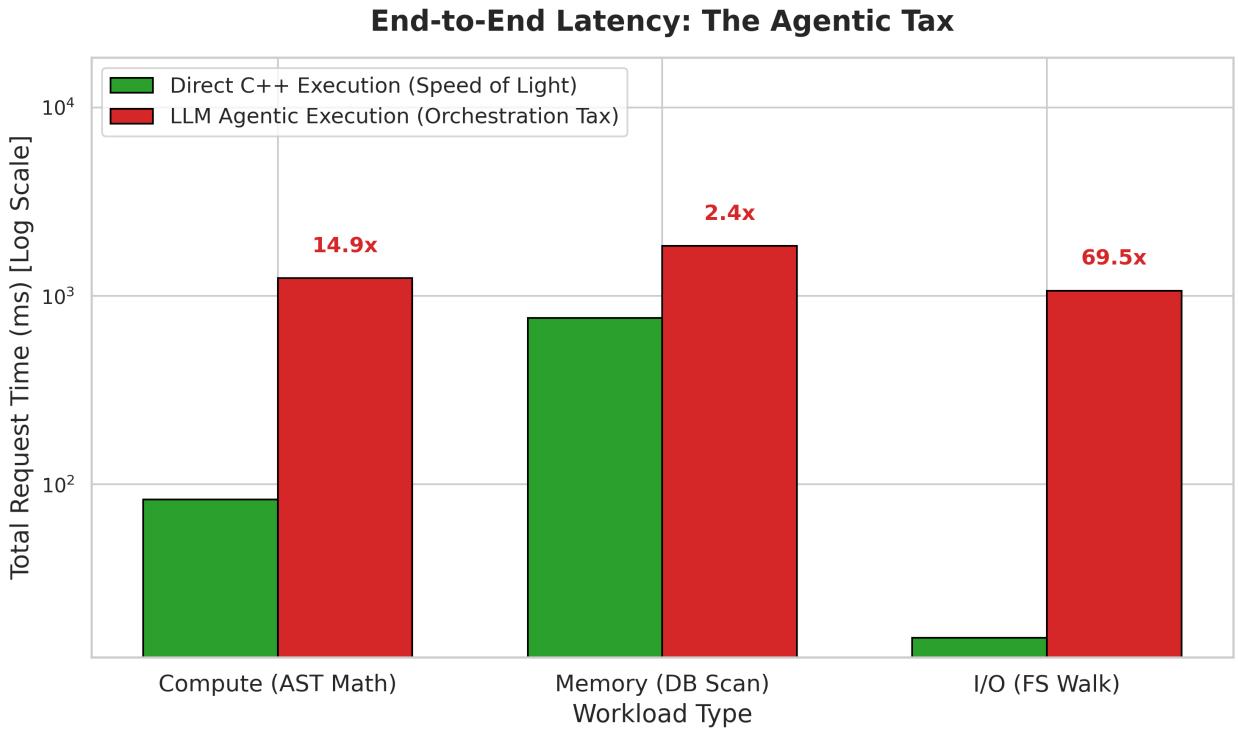


Figure 1: End-to-End Latency Comparison: Direct C++ Execution vs. LLM Agentic Orchestration. Note the logarithmic scale on the Y-axis. The red annotations indicate the multiplicative latency penalty (the "Agentic Tax") for each workload class.

#### Analysis of Orchestration Overhead

The data immediately reveals that the "Agentic Tax" is not a proportional scalar, but rather a massive, fixed-cost penalty that disproportionately punishes highly optimized, lightweight tools.

For the I/O-bound filesystem walk, the native C++ kernel executes in approximately 15ms. However, routing this identical task through the LLM orchestrator results in a staggering 69.5× latency slowdown. This massive penalty is a compound tax: the GPU time required to generate the JSON response, combined with the CPU-bound time required to tokenize the prompt, parse the Python output, and format the final string, vastly eclipses the time spent actually traversing the filesystem. Similarly, the compute-bound Abstract Syntax Tree (AST) mathematical evaluator suffers a 14.9× penalty.

Conversely, the memory-bound database lookup exhibits a significantly lower multiplier of 2.4×. This aligns with Amdahl's Law: because the forced full-table scan requires over 2.5 billion CPU cycles and natively consumes ~ 764ms, the fixed LLM orchestration and GPU generation costs account for a proportionally smaller fraction of the total execution

time.

These findings confirm our primary hypothesis: for the vast majority of agentic tasks, execution time is entirely dominated by the LLM framework's orchestration overhead rather than the tool kernel's execution time [1–3]. This "masking effect" forces us to re-evaluate how we optimize agentic systems at the hardware level.

### 4.2 Compounding Overhead in Multi-Step Workflows

While isolated, single-step tool invocations incur a massive fixed penalty, real-world autonomous agents execute multi-step ReAct (Reasoning and Acting) loops. In these workflows, the agent must coordinate between diverse tool classes, digesting the output of one tool to inform the inputs of the next.

To evaluate the microarchitectural cost of this coordination, we deployed a "Gauntlet" prompt requiring the Qwen-2.5-7B agent to sequentially execute the I/O walker, the database query, and the math evaluator within a single, continuous context window. We isolated the CPU instructions executed by the Python LLM orchestrator (tokenization, string decoding, and JSON parsing) versus the instructions executed by the compiled C++ kernels.

#### The ReAct Execution Model

To contextualize the hardware telemetry, we must first define the execution model of the LLM orchestrator. A single "Step" in our architecture is defined as one complete forward pass of the LLM orchestration layer followed by the optional execution of a C++ kernel. During the Gauntlet payload, the system executed four distinct steps:

- **Step 1 (Initialization & I/O):** The orchestrator tokenizes the initial system prompt and schemas, and the LLM routes to the `walk_directory` kernel.
- **Steps 2 & 3 (Context Accumulation):** The `stdout` results from previous tools are appended to the context window. The LLM re-processes this expanding history to route the `query_database` and `evaluate_math_file` kernels, respectively.
- **Step 4 (Termination):** Recognizing all tasks are complete, the LLM generates a plain-text summary. No C++ kernels are invoked, breaking the loop.

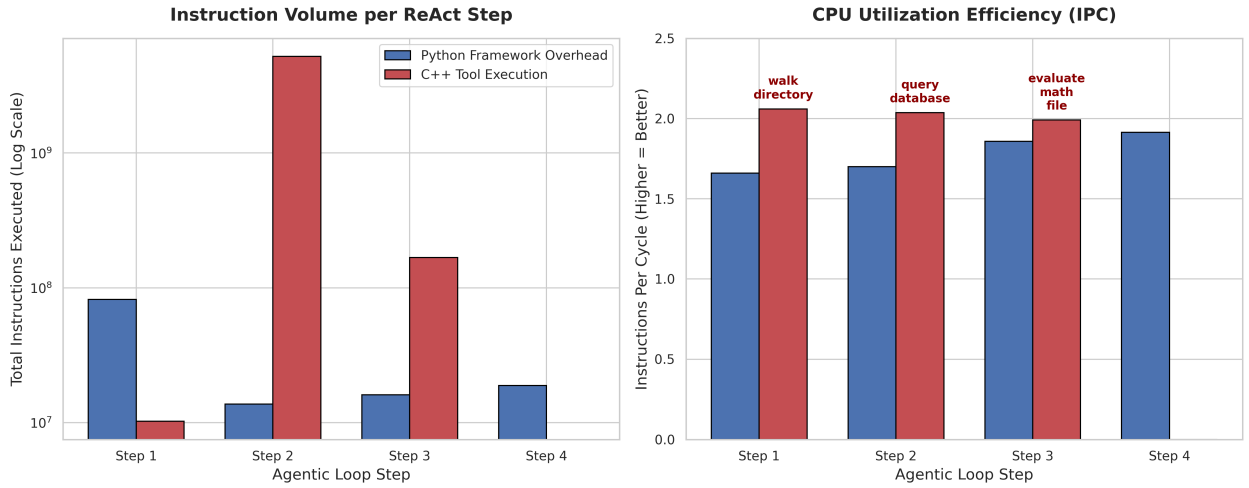


Figure 2: Instruction Volume and Efficiency across a Multi-Step Agentic Loop. (Left) The logarithmic volume of instructions executed by the framework versus the tools. (Right) The Instructions Per Cycle (IPC) demonstrating the CPU efficiency gap between native tools and Python orchestration.

#### Analysis of Microarchitectural Trends

As illustrated in Figure 2, tracking the microarchitectural telemetry across these four steps reveals two distinct phenomena within the Python framework and the C++ execution layer.

**1. Framework Overhead: The Cold Start vs. Context Accumulation.**
The orchestration overhead is not static across an agentic loop. At Step 1, the Python framework executes a massive $\sim$ 82 million instructions. This represents a one-time "Cold Start Penalty"—the CPU-bound effort required to parse and load the complex JSON schema definitions into memory. Once initialized, the steady-state loop overhead across Steps 2, 3, and 4 drops to a baseline of 13.7 million instructions. However, because the LLM is stateless, the framework instruction volume grows linearly from this baseline (peaking at 18.8 million instructions) with each successive tool call as the Python orchestrator must re-serialize a progressively expanding conversation history.

Interestingly, Figure 2 (Right) shows the Framework's Instructions Per Cycle (IPC) actually improving from 1.65 (Step 1) to 1.91 (Step 4). This reflects an efficiency shift: the initial tokenization relies on memory-heavy lookups (causing CPU pipeline stalls), whereas later steps rely heavily on simpler, predictable string concatenation that keeps the CPU pipeline fuller.

**2. Tool Execution: Payload-Dictated Bottlenecks.**
Conversely, the C++ execution metrics are entirely independent of the step count and dictated exclusively by the payload. The Instruction Volume (Figure 2, Left) wildly fluctuates from 10 Million instructions for the basic I/O walk to over 5 Billion instructions for the forced database scan.

Furthermore, while the Python framework struggles with efficiency (the "Language Barrier"), the compiled C++ kernels operate tightly near the theoretical maximum throughput of the CPU pipeline ($\sim$2.0 IPC). However, minor fluctuations reveal specific microarchitectural bottlenecks: the IPC of the DB scan remains remarkably high (2.03) because the CPU's hardware L3 Prefetcher successfully streams the sequential table scan into cache. Conversely, the AST math evaluator's IPC dips slightly (1.99) due to the massive volume of `if/else` branching required to parse nested math strings, causing hundreds of thousands of pipeline-flushing branch mispredictions.

Ultimately, these metrics confirm that LLM coordination overhead is a compounding tax that scales with agent autonomy, perpetually dragging down overall system efficiency despite highly optimized native tooling [1, 5].

### 4.3 Microarchitectural Optimizations and the Orchestration Masking Effect

To rigorously evaluate the potential for hardware-accelerated tool calls, we targeted the three fundamental system bottlenecks—compute, memory, and I/O—by applying classical microarchitectural optimizations to the execution kernels. Our objective was to quantify whether kernel-level efficiency gains propagate to end-to-end system latency, or if they are absorbed by the orchestration layer overhead defined in previous sections.

#### 4.3.1 Compute-Bound Execution: The Vectorization Wall

In our first optimization experiment, we compiled the Abstract Syntax Tree (AST) mathematical evaluator using Advanced Vector Extensions (`AVX2`). The hypothesis was that 256-bit wide SIMD (Single Instruction, Multiple Data) registers would accelerate numerical parsing throughput. However, hardware telemetry revealed that agentic AST evaluation is intrinsically scalar and branch-bound.

As illustrated in Figure 3, the parsing logic relies heavily on recursive control-flow divergence and pointer-chasing memory access patterns, which effectively neutralized the compiler's auto-vectorization capabilities. Consequently, kernel-level instruction throughput remained static (IPC $\approx$ 1.99), and the tool execution cycles showed a negligible 0.6% improvement (dropping from $84.1 \times 10^6$ to $83.6 \times 10^6$ cycles). This outcome reflects a limitation of simple, compiler-driven SIMD on branch-heavy recursive parsers; achieving SIMD speedups for string parsing typically requires algorithmic restructuring and explicit SIMD techniques, as demonstrated by high-performance parsers such as `simdjson` [4].

#### 4.3.2 Memory-Bound Execution: Latency Hiding via Software Prefetching

To evaluate memory bottlenecks, we transitioned to a full-table database scan kernel, bypassing indexes to intentionally thrash the Last-Level Cache (LLC). We introduced a hardware-aware optimization by injecting software prefetch instructions (`__builtin_prefetch`) with a lookahead distance of 32 elements. This explicitly instructs the CPU's

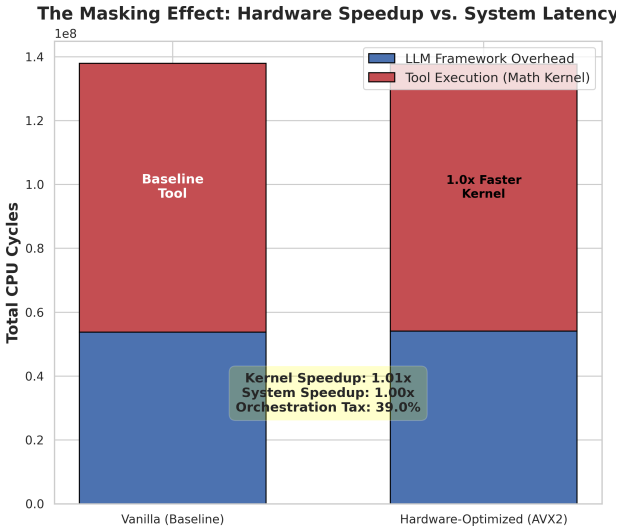


Figure 3: Microarchitectural analysis of the AST evaluator kernel. The data demonstrates a negligible cycle reduction and stagnant IPC, highlighting the failure of auto-vectorization on branch-heavy parser logic.

memory controller to fetch heap data into the L1 cache before the execution pipeline requires it.

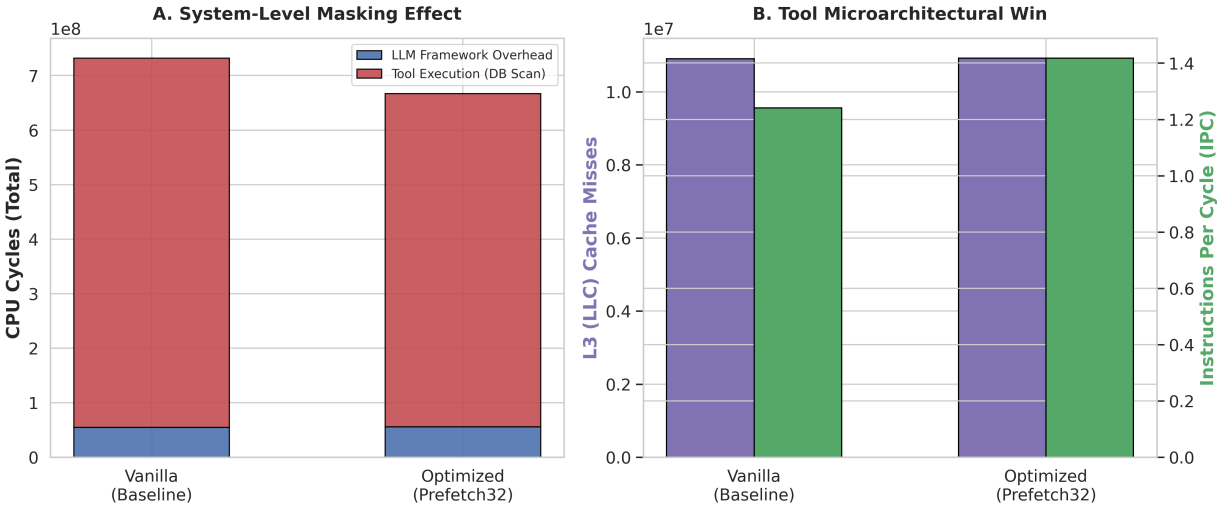


Figure 4: Impact of software prefetching on the database scan kernel. (Left) The end-to-end masking effect. (Right) The microarchitectural victory, showing a 14.2% IPC improvement and successful latency hiding despite constant L3 cache misses.

The profiling data, detailed in Figure 4, confirmed a significant microarchitectural victory. By converting synchronous CPU stalls into asynchronous background memory loads, the prefetching kernel successfully decoupled memory latency from instruction execution. While the absolute volume of LLC misses remained physically constant due to the cold-cache nature of the scan ($\sim 10.9 \times 10^6$ misses), the penalty of those misses was neutralized. This resulted in a denser execution pipeline, raising the IPC by 14.2% (from 1.241 to 1.417) and completing the kernel workload in nearly 10% fewer CPU cycles (saving $\sim 66 \times 10^6$ core cycles). Factoring in concurrent OS scheduling overhead and the CPU base clock, this translated to a roughly 50ms reduction in wall-clock execution for the native kernel..

#### 4.3.3 The System-Level Reality: The Masking Effect

Despite the 14.2% microarchitectural efficiency gain achieved in the memory-bound workload, the end-to-end latency metrics expose a severe limitation in current agentic architectures. The ~ 66 million cycle reduction at the C++ kernel level translated to roughly 50ms of real-world time savings. However, the total end-to-end request latency was approximately 6.3 seconds. This overarching latency is a compound penalty, dominated heavily by the GPU token generation time during the forward pass, coupled with the CPU-bound Python JSON serialization and routing overhead..

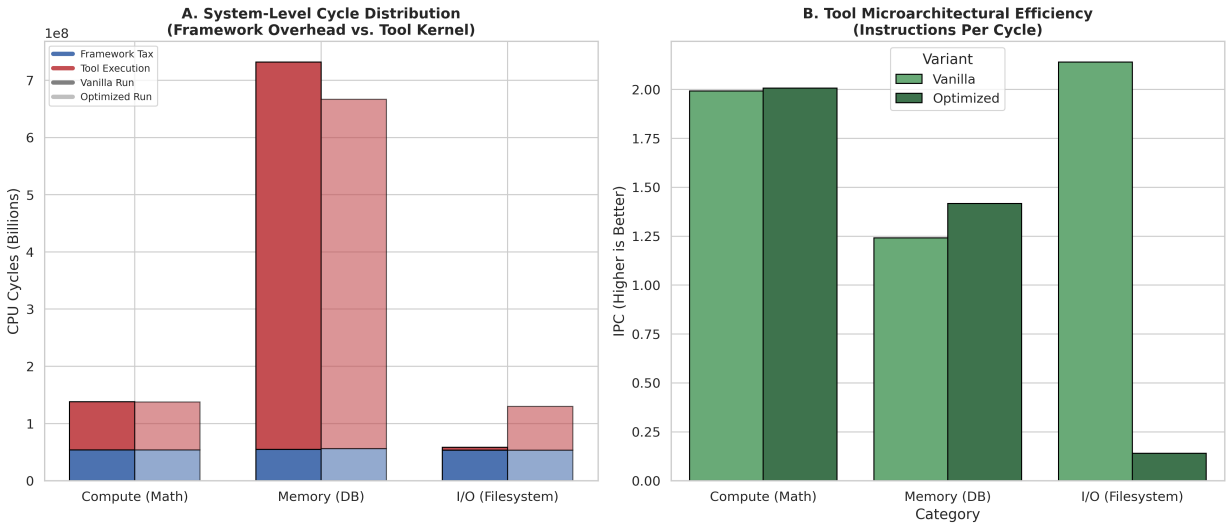


Figure 5: System-level cycle distribution and tool efficiency across all optimized computing domains. The contrasting panels demonstrate the “Masking Effect”, where substantial kernel-level IPC gains (Right) fail to impact the massive framework orchestration cycles (Left).

Consequently, a highly tuned, hardware-aware memory optimization yielded a sub-1% improvement in end-to-end system speedup. Parallel experiments targeting I/O-bound bottlenecks via OpenMP concurrency exhibited similar masking phenomena. As visualized across all three computing domains in Figure 5, the empirical evidence points to a definitive "Masking Effect": the execution bloat introduced by the orchestration framework is so massive that it effectively renders kernel-level hardware optimizations statistically invisible to the end user.

## 5 Limitations and Discussion

While our decoupled architecture and telemetry pipeline successfully isolated the microarchitectural overhead of agentic orchestration, several methodological constraints and hardware realities limit the generalizability of our findings.

### 5.1 Methodological Constraints in Hardware Profiling

Our reliance on the Performance Application Programming Interface (PAPI) introduced a specific profiling artifact during the evaluation of concurrent workloads. The high-level PAPI API tracks hardware telemetry exclusively on the calling thread. In Experiment 5 (I/O-Bound Directory Walker), we attempted to mask I/O latency using OpenMP task parallelism. However, the recorded IPC plummeted to an anomalous 0.14.

Analysis of the execution model reveals this is a profiling blind spot rather than a true systemic degradation. The master thread, which initialized PAPI, entered a spin-wait state at the OpenMP barrier, executing minimal instructions while accumulating massive cycle counts. The telemetry failed to capture the cumulative instruction throughput of the background worker threads. Consequently, our framework accurately captures scalar execution efficiency but requires future modification (such as thread-local PAPI event sets) to accurately quantify multi-threaded kernel efficiency.

### 5.2 Concurrency Limits and Negative Scaling

Beyond the profiling artifact, the OpenMP I/O optimization exhibited textbook negative scaling. The optimized kernel required 15× more CPU cycles to complete the same file-system walk as the single-threaded baseline. Profiling revealed that aggregating the file count via a global atomic variable (`std::atomic`) induced severe L1 cache lock contention. The worker threads spent the majority of their execution time stalled, fighting for exclusive cache-line access. Furthermore, parallelizing disk reads for a small directory hit immediate serialization bottlenecks within the OS kernel. This demonstrates that agentic tool frameworks cannot blindly rely on multithreading to solve I/O bottlenecks without implementing highly tuned, lock-free, thread-local data structures and ensuring massive payload sizes to justify the thread-pool overhead [3].

### 5.3 Static Workloads vs. Real-World Variance

To ensure deterministic execution and pristine cycle counting, generation sampling was disabled (`do_sample=False`) and static context payloads were utilized. In production environments, LLMs frequently hallucinate tool arguments, requiring multi-turn error-correction loops [7]. This dynamic variance would exponentially increase the "Agentic Tax" measured in our study, meaning our results represent a best-case lower bound for orchestration overhead [8].

## 6 Conclusion and Future Work

As the artificial intelligence industry aggressively pivots toward autonomous, multi-step agentic workflows, the underlying systems architecture must adapt to support massive tool-calling concurrency [9]. In this paper, we introduced a decoupled profiling framework to isolate and quantify the "Agentic Tax"—the microarchitectural overhead imposed by LLM orchestration layers.

Our empirical evaluation of compute, memory, and I/O-bound tool kernels yields a definitive conclusion: the current paradigm of agentic execution is crippled by an orchestration bottleneck. We demonstrated that for lightweight tools, the combined penalty of GPU token generation and CPU-bound Python JSON serialization can induce a 69.5× latency slowdown compared to native execution. These results align with CPU-centric profiling of agentic AI frameworks, where host-side tool processing and orchestration frequently dominate end-to-end latency [3].

Crucially, our microarchitectural interventions proved that classical hardware optimizations at the tool level are rendered statistically irrelevant by this overarching tax. Even when successfully deploying software prefetching to hide RAM latency and increase kernel IPC by 14.2%, the end-to-end system speedup remained below 1%. We term this the **Masking Effect**: the execution bloat of the orchestration framework is so massive that it effectively swallows any microarchitectural victories achieved within the tool kernels.

### 6.1 Future Work

Based on these findings, future systems research must pivot away from optimizing tool-execution kernels and focus aggressively on minimizing the orchestration layer. We propose three critical avenues for future work:

1. **High-Throughput Inference Engines:** Transitioning away from naive, synchronous generation pipelines to continuous batching frameworks (e.g., vLLM) that utilize PagedAttention to minimize the GPU generation latency portion of the Agentic Tax [1, 2].

2. **Compiled Orchestrators and Zero-Copy Routing:** Migrating the orchestration logic from Python to compiled languages (Rust/C++). Furthermore, replacing standard JSON libraries with SIMD-accelerated parsers (e.g., `simdjson`) or fast native JSON libraries (e.g., `orjson`) can drastically reduce the CPU cycles wasted on serialization and dictionary lookups [4].

3. **Standardized Binary Protocols:** The reliance on string-based JSON for LLM-to-tool communication is fundamentally inefficient. Future research should explore fine-tuning LLMs to output structured, in-memory binary formats (such as Apache Arrow) to allow tools to execute directly on the generation buffer without intermediate parsing.

Ultimately, until the orchestration layer is treated as a high-performance networking data plane, the theoretical potential of hardware-accelerated agentic tools will remain masked by the framework that invokes them.

## 7 References


[1] Efficient LLM Serving for Agentic Workflows: A Data Systems Perspective. https://arxiv.org/abs/2603.16104v1

[2] Sutradhara: An Intelligent Orchestrator-Engine Co-design for Tool-based Agentic Inference. https://arxiv.org/abs/2601.12967

[3] A CPU-Centric Perspective on Agentic AI. https://arxiv.org/abs/2511.00739

[4] Parsing Gigabytes of JSON per Second. https://arxiv.org/abs/1902.08318

[5] Act While Thinking: Accelerating LLM Agents via Pattern-Aware Speculative Tool Execution. https://arxiv.org/abs/2603.18897v1

[6] Characterizing and Optimizing LLM Inference Workloads on CPU-GPU Coupled Architectures. https://arxiv.org/abs/2504.11750v1

[7] AI-NativeBench: An Open-Source White-Box Agentic Benchmark Suite for AI-Native Systems. https://arxiv.org/abs/2601.09393v1

[8] ToolCaching: Towards Efficient Caching for LLM Tool-calling. https://arxiv.org/abs/2601.15335

[9] Aragog: Just-in-Time Model Routing for Scalable Serving of Agentic Workflows. https://arxiv.org/abs/2511.20975

[10] ReAct: Synergizing Reasoning and Acting in Language Models. https://doi.org/10.48550/arXiv.2210.03629

# Appendices

## A Member Contributions

- **Kunpeng Zhang:**
  - **Code:** Engineered the three primary C++ execution kernels (AST Math, DB Scan, FS Walk) and synthesized static workloads. Integrated PAPI with the Linux PMU for hardware telemetry. Implemented hardware optimizations (AVX2, prefetching, OpenMP) and developed the Python-to-C++ integration pipeline.
  - **Report:** Authored the *Project Methodology* section, detailing system decoupling, microarchitectural telemetry integration, and kernel design.
  - **Presentation:** Presented the methodology, C++ tool implementation, and the technical details of the hardware optimizations applied.
- **TJ Grewal:**
  - **Code:** Architected the decoupled benchmarking harness to isolate the Python GIL from the compiled C++ execution environment and established the "direct mode" control baseline.
  - **Report:** Primary author for the *Abstract*, *Introduction*, and *Background* sections. Conducted the literature review defining the Agentic Tax and modern LLM bottlenecks.
  - **Presentation:** Presented the project motivation, background on ReAct frameworks, and the conceptual system architecture.
- **Fazal Rehman:**
  - **Code:** Designed the data extraction and visualization scripts to synthesize raw hardware telemetry into latency and IPC graphs. Managed repository documentation and setup instructions.
  - **Report:** Primary author for the *Evaluation Results*, *Limitations*, and *Conclusion* sections. Formulated the "Masking Effect" theory and diagnosed the OpenMP profiling artifacts.
  - **Presentation:** Presented the empirical findings, data visualizations, and concluded with the implications for future agentic architectures.

## B Documentation and Code Instructions

*Note: For exhaustive documentation, including repository structure, raw data handling, and edge cases, please refer to the* `README.md` *located in the root of the submitted code repository. All experimental results (.csv) and generated figures (.png) have been pre-computed and retained in the* `results/` *directory for immediate review.*

### B.1 Repository Access

- **Source Code:** https://github.com/AndyZzzZzzZzz/beyond-gpu-kernels

### B.2 System Prerequisites & Environment Setup

- **Hardware:** GPU with ≥ 32GB VRAM (e.g., NVIDIA V100) to host the Qwen-2.5-7B model. Intel CPU with Performance Monitoring Unit (PMU) enabled.
- **OS & Permissions:** Linux OS. PAPI requires kernel-level access to hardware performance counters. You must enable `perf` access:

```
sudo sysctl -w kernel.perf_event_paranoid=-1
```

- **Dependencies:** `g++` (C++17), `libsqlite3-dev`, and `libpapi-dev`.

**Python Environment Setup:**

```
conda env create -f environment.yml
conda activate beyond-gpu
pip install torch transformers fastapi uvicorn requests pypapi
pip install matplotlib seaborn pandas numpy accelerate
```

### B.3 Compiling the Native C++ Kernels

To capture accurate microarchitectural telemetry, both the baseline and hardware-optimized versions of the C++ tools must be compiled with strict hardware flags from the project root:

```
# 1. Compute-Bound (Baseline vs. AVX2 SIMD)
g++ -O3 -march=native tools/calculator/evaluator.cpp \
    -o tools/calculator/eval_baseline -lpapi -static-libstdc++
g++ -O3 -march=native -mavx2 tools/calculator/evaluator.cpp \
    -o tools/calculator/eval_avx2 -lpapi -static-libstdc++

# 2. Memory-Bound (Baseline vs. Software Prefetching)
g++ -O3 -march=native tools/db_retrieval/db_lookup_baseline.cpp \
    -o tools/db_retrieval/db_lookup_baseline -lsqlite3 -lpapi
g++ -O3 -march=native tools/db_retrieval/db_lookup_scan_prefetch32.cpp \
    -o tools/db_retrieval/db_lookup_scan_prefetch32 -lsqlite3 -lpapi

# 3. I/O-Bound (Baseline vs. OpenMP Task Parallelism)
g++ -O3 -march=native tools/io_walker/io_walker_baseline.cpp \
    -o tools/io_walker/io_walker_baseline -lpapi
g++ -O3 -march=native -fopenmp tools/io_walker/io_walker_omp.cpp \
    -o tools/io_walker/io_walker_omp -lpapi
```

### B.4 Execution Pipeline & Reproduction

The testing harness strictly decouples the LLM orchestrator from the execution telemetry. Run the server in one terminal and execute the tests in another.

```
# Step 1: Start the Orchestration Server (Terminal 1)
cd src
python model_loader.py

# Step 2: Generate Deterministic Stress Payloads (Terminal 2)
cd workloads/single_step
python generate_db_stress.py
python generate_fs_stress.py
python generate_math_stress.py

# Step 3: Run the Agentic Evaluation Suite (Terminal 2)
# Compares Direct C++ Execution vs. the LLM "Agentic Tax"
python test_single_direct.py
python test_single_llm.py

# Step 4: Run Microarchitectural Optimization Tests (Examples)
python test_exp3_llm_vanilla.py && python test_exp3_llm_optimized.py
python test_exp4_db_vanilla.py && python test_exp4_db_optimized.py

# Step 5: Generate Visualizations
cd ../../
python plots.py # Outputs saved to the /results directory
```